OPENING — SET THE STAGE

The problem in one sentence

Today's cloud systems are reactive: they only add servers after CPUs are already overloaded — but new servers take 2–5 minutes to boot, so by the time they're ready, users have already experienced failures. We solve this by predicting traffic spikes 15 minutes in advance using an LSTM neural network, so servers are ready before the load arrives.

WHY THIS MATTERS TO UNISYS

The business case

SLA violation cost

Penalties + churn

Every violated SLA costs money and trust

Over-provisioning waste

Idle $ burned

Running servers you don't need is just waste

Benchmark improvement

~33.7%

CPU utilisation gain vs threshold-based (Zhang et al. 2025)

Cost reduction

~32.7%

Vs reactive autoscaling (Zhang et al. 2025, ACM)

Unisys manages cloud infrastructure for enterprise clients at scale. Every percentage point improvement in autoscaling efficiency — fewer SLA violations, less idle capacity — translates directly to client satisfaction and cost savings. This is exactly the problem space our project addresses, with a reproducible, open, platform-independent approach that works on any cloud — not just AWS or GCP.

TECHNICAL ARCHITECTURE

How the system works — 3 layers

Layer 1: Brain	Forecasting layer — An LSTM neural network (2 layers, 128 units, Adam optimiser at 0.001 LR) takes a sliding window of the last 20–30 minutes of request rate data and predicts load for the next 15 minutes. Built with Python, Keras, and TensorFlow.
Layer 2: Manager	Decision layer — Takes the LSTM's prediction and current cluster state. Applies a cost-aware penalty function that penalises both over-provisioning (wasted idle servers) and under-provisioning (SLA violation risk). Outputs a scaling action: scale up, scale down, or hold.
Layer 3: Cloud	Simulation layer — Models the cloud cluster, applies a 2-minute startup delay for new servers (reflecting real provisioning time), routes requests, and logs SLA violations and resource usage. Built using Python classes modelling Server and Cluster objects.

DATA AND VALIDATION

Why our results are credible

Alibaba Cluster Trace 2018

We train and test on real production workload data from Alibaba's data centres — publicly released for academic research. It contains CPU usage, request rates, and resource consumption data exhibiting exactly the patterns LSTM must learn: daily cycles, weekly cycles, and irregular bursty spikes. This is not synthetic data we made up.

Three-way comparison — fair evaluation

We run all three systems on the exact same simulation: (1) our LSTM predictive scaler, (2) reactive threshold-based autoscaler (conceptually identical to Kubernetes HPA — the industry standard), and (3) ARIMA forecaster (classical statistical baseline). Measuring five metrics: SLA violation rate, over-provisioning waste, under-provisioning exposure, cost score, and forecast RMSE.

KNOWN LIMITATIONS — ANSWERED HONESTLY

Drawbacks and how we address them

Limitation 1: Simulation, not real infrastructure

We use a Python-based simulation rather than real AWS or Kubernetes clusters — because running live experiments costs money and requires production access. This is standard academic methodology: the Lanciano et al. 2021 IEEE/ACM paper (our primary reference) also uses a simulated testbed. Future deployment pathway: expose our LSTM predictions as a custom metric via a Python service and use KEDA (Kubernetes Event Driven Autoscaler) to act on those predictions — a real, documented path to production.

Limitation 2: Single LSTM model, not an ensemble

Zhang et al. (2025, ACM) use an ensemble of LSTM + XGBoost + Prophet. We deliberately scope to a single LSTM — which is still enough to outperform reactive baselines and is academically honest about our scope as a four-person beginner team with 4 months. Ensemble extension is documented as explicit future work.

Limitation 3: Dataset from 2018

The Alibaba trace is from 2018. However, it remains the standard open benchmark in autoscaling research — freely available, widely cited, and containing the traffic patterns (daily cycles, weekly patterns, random spikes) that are still representative of modern workloads.

SCALABILITY AND PRODUCTION PATH

How this goes from research to real systems

Why it scales

The LSTM model is trained offline on historical data and runs inference in milliseconds. The decision layer is pure Python logic — stateless and fast. The architecture is platform-independent: any cloud (AWS, Azure, GCP, on-premise) can feed request rate data in and receive a scaling decision out. No proprietary lock-in.

Production deployment pathway

Step 1 — wrap the LSTM inference in a REST microservice. Step 2 — connect it to KEDA (Kubernetes Event Driven Autoscaler) as an external scaler. Step 3 — KEDA polls our service every 5 minutes and scales pods based on predicted load. This is a real, documented architecture that banks, hospitals, and government systems running private infrastructure (who cannot use AWS Predictive Scaling) can adopt.

WHAT DIFFERENTIATES US

Why this matters vs what already exists

Anticipated question: "AWS already does predictive autoscaling — why build this?"

AWS Predictive Scaling and Google GKE ML scaling are proprietary black boxes. You cannot see how they work, modify them for your workload, or deploy them outside their cloud. They are also unavailable to organisations running private infrastructure — banks, hospitals, government systems, defence contractors like Unisys clients.

Our system is transparent, open-source, and platform-independent. Any organisation can audit every number, every decision, and every graph. This is what academic research is supposed to produce — the reproducible, open version the entire community can build on.

Open source Platform independent Reproducible results Real production data Peer-reviewed methodology

ANSWERING THE PROFESSOR'S QUESTION

On "building cloud infrastructure"

The professor likely asked whether you've deployed this on real cloud infrastructure (AWS/Kubernetes) rather than simulating it. The answer is: not yet — and here is why that's acceptable and what the next step looks like.

Why simulation is academically valid

IEEE/ACM and top venues regularly validate autoscaling approaches through simulation. The Lanciano et al. 2021 paper — published at IEEE/ACM UCC — also uses a simulated testbed. Our simulation models realistic server startup delays and request queuing, making it academically sound. The transition to real infrastructure is documented as future work, which is exactly what a professor wants to hear.

The concrete next step (what to say)

"To deploy on real infrastructure, we expose our LSTM inference as a Python REST service and connect it to KEDA — the Kubernetes Event Driven Autoscaler. KEDA polls our prediction service every N minutes and uses the output to scale pods. This is a real deployment pathway, not theoretical — it's documented in the KEDA external scalers specification. We acknowledge simulation as a current limitation and describe this path explicitly in our conclusion."